

GSM: EFFICIENT LANGUAGE MODELING WITH SHARED GLOBAL STATE

Yunao Zheng¹ Bin Wen^{2,*†} Xiaojie Wang^{1,†}

Kaiyu Jiang Xuanyu Zheng Changyi Liu Hongyi Fu
Jianxiong Wang Tianke Zhang Haonan Fan Yingxin Li
Jiankang Chen Xu Wang Tingting Gao Han Li

ABSTRACT

Efficient language models must reduce not only the cost of individual accesses to past context but also the overhead of repeatedly selecting and processing historical information across layers. We introduce the Global State Model (GSM), a causal encoder-decoder architecture that concentrates the selection and aggregation of long-range information in the encoding stage. Through multiple stages of history retrieval, the encoder progressively incorporates long-range information into representations at recent positions, forming a shared state with a fixed window size. Each decoder layer accesses this same state using queries updated from the preceding layer, preserving computational depth while avoiding repeated construction of historical key-value (KV) representations and long-range indexing. As a result, neither the decoder’s per-step attention cost nor its KV cache size grows with the history length. Experiments show that GSM improves computational efficiency and reduces cache overhead while maintaining model performance and the ability to use long-range information, offering a shared-state architecture for efficient language modeling.

1 INTRODUCTION

The practical utility of language models depends on two interdependent objectives: achieving strong task performance and delivering that performance at low cost. Research increasingly shows that data quality, training methods, and parameter count jointly determine model performance Penedo et al. (2024); Ouyang et al. (2022); Hoffmann et al. (2022). Architectural design, in turn, addresses how to achieve these objectives with less computation while preserving the necessary information pathways and computational depth.

A central challenge is the cost of accessing historical information. Global attention allows each position to query its entire causal history, but its computational cost grows quadratically with sequence length Vaswani et al. (2017). Recent efficient architectures address this cost through two representative approaches. Hybrid architectures use linear attention for sequence processing in most layers while retaining a small number of global attention layers. For example, Kimi K3 combines Kimi Delta Attention with gated multi-head latent attention (MLA), allowing recurrence over a fixed-size state and explicit long-range retrieval to complement each other Kimi Team (2026). Sparse attention instead limits the number of historical entries accessed by each query and further reduces overhead through key-value (KV) compression and reuse, as exemplified by DeepSeek-V4 DeepSeek-AI (2026a).

These approaches primarily reduce the cost of processing historical information within individual layers, but do not fully eliminate the overhead of repeatedly representing, selecting, and accessing that information across model depth. This raises a more fundamental question: must every layer independently construct and maintain its own representation of history? A deep network must satisfy two essential conditions. First, its state representations must preserve access to task-relevant historical information so that distant information remains accessible to subsequent computation. Second,

¹ Beijing University of Posts and Telecommunications. ² Kuaishou Technology. ^{*} Project Leader. [†] Corresponding authors: Xiaojie Wang and Bin Wen.

subsequent computation must depend on previously formed representations, allowing information to be progressively combined and refined across layers. The former ensures information accessibility, while the latter preserves computational depth. Neither condition, however, requires each layer to maintain an independent historical state. Provided that a shared state retains the historical information needed for subsequent computation and that queries in deeper layers are derived from representations in earlier layers, different layers can perform distinct retrieval and computation over the same historical state while preserving progressive refinement across layers Brandon et al. (2024).

DeepSeek-V4.1-Flash already makes partial use of this property. It adopts an encoder–decoder architecture in which the encoder constructs historical representations that the decoder subsequently reuses, avoiding the need to construct a complete global KV representation independently for every decoder layer. However, sharing historical representations is not equivalent to sharing a state in which information has already been selected and aggregated. Decoding still requires indexing and sparse retrieval over historical representations based on the current query, as well as a local window to retain recent information DeepSeek-AI (2026b). In other words, the construction of historical representations can already be shared across layers, but the decision about which historical information is needed for subsequent computation is still made repeatedly during decoding. A natural next step is to move this process into the encoder as well: beyond constructing historical representations, the encoder can progressively select and aggregate the information needed by subsequent computation, allowing the decoder to operate directly on a fixed-size shared state that can be reused across layers.

Motivated by this observation, we propose the Global State Model (GSM), a causal encoder–decoder architecture. Across multiple stages, the encoder combines the representation at the current position with the accumulated retrieval output to progressively aggregate sparsely selected historical information and fuse it into the hidden representation at that position. Representations at a fixed number of recent positions form a shared state window that is updated as the sequence advances. All decoder layers access the same KV representation derived from this state, with each layer generating new queries from the output of the preceding layer. This design concentrates the selection and aggregation of historical information in the encoding stage while retaining progressive representation refinement in the decoding stage. GSM thus shifts the focus of efficient context modeling from reducing the cost of history access at each layer to constructing a shared state that can be reused by subsequent deep computation.

Our experiments support this design. Across multiple parameter scales, GSM achieves downstream task performance comparable to that of baseline models built on DeepSeek-V4.1-Flash. In long-context experiments, GSM successfully performs needle-in-a-haystack retrieval at context lengths of up to 64K using a decoder shared state window of only 256 positions. This demonstrates that, under the evaluated setting, a fixed-size decoder state can support the use of distant information. Its long-context perplexity also follows a trend similar to that of the baseline as sequence length increases. Meanwhile, GSM reduces the decoder’s computational complexity for interactions across sequence positions and its cache requirements, yielding higher speed and lower GPU memory usage in practical efficiency benchmarks. These results show that concentrating the selection and aggregation of long-range information in shared state construction can reduce the cost of long-context computation while maintaining performance on the evaluated tasks.

2 RELATED WORK

2.1 FROM SHARED HISTORICAL REPRESENTATIONS TO SHARED STATES

The decoder in DeepSeek-V4.1-Flash combines long-range sparse retrieval with local-window attention, providing access to distant information while retaining recent representations. GSM also retains representations at recent positions, but incorporates retrieved historical information into these representations during encoding and shares the resulting KV across layers. Recent information and retrieved long-range information are therefore represented within a single state window, over which the decoder performs attention.

To characterize the difference in computational cost, let N denote the history length, L_D the number of decoder layers, K the number of entries accessed through long-range sparse retrieval, W the local window size, P the candidate pool capacity, J the number of Reindex layers, and M the shared state size in GSM. Let d_I and d denote the effective computational widths of index scoring and

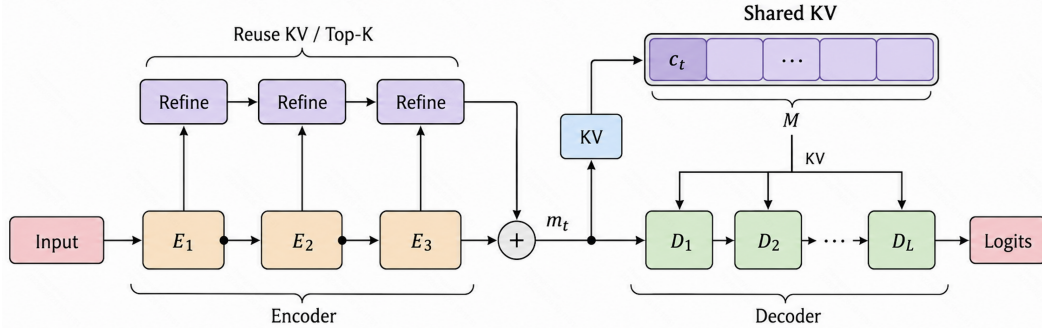


Figure 1: Overview of the GSM architecture. The encoder progressively refines historical information, while the decoder layers share a single fixed-size KV memory.

main attention, respectively. In DeepSeek-V4.1-Flash, the first Full layer in the decoder indexes the history, the subsequent J Reindex layers reselect entries from the candidate pool, and the remaining Reuse layers reuse the selection results. The per-token attention costs of the decoders are

$$T_{\text{DS},D}^{\text{decode}} = \mathcal{O}(Nd_I + JPd_I + L_D(K + W)d), \quad (1)$$

$$T_{\text{GSM},D}^{\text{decode}} = \mathcal{O}(L_DMd). \quad (2)$$

GSM removes the decoder indexing term that grows with history length and limits each layer to accessing M entries. When $M < K + W$, it also reduces the interaction cost of main attention.

For cache storage, let d_{KV} denote the effective storage width of a KV entry. Assuming that window caches are retained directly during ongoing decoding, the KV cache sizes required by the decoders are

$$C_{\text{DS},D} = \mathcal{O}(Nd_{\text{KV}} + L_DWd_{\text{KV}}), \quad (3)$$

$$C_{\text{GSM},D} = \mathcal{O}(Md_{\text{KV}}). \quad (4)$$

GSM therefore provides substantial GPU memory savings for long input sequences.

2.2 CONNECTIONS TO LINEAR ATTENTION

Linear attention summarizes historical information in a fixed-size state Katharopoulos et al. (2020). For example, the update in DeltaNet can be abstracted as Schlag et al. (2021); Yang et al. (2024)

$$S_t^{\text{lin}} = A(x_t)S_{t-1}^{\text{lin}} + \beta_t k_t v_t^\top, \quad (5)$$

where newly written information is generated primarily from the input at the current position. Such linear attention methods can use historical information through a recurrent state. Once the original history has been compressed, however, subsequent positions can no longer directly retrieve individual entries from it.

GSM differs in that the process of writing to the shared state retains access to an explicit representation of the history. At each stage, it can generate a new query from the current hidden representation and the accumulated retrieval output, and access the full causal history again. Subsequent retrieval can thus depend on information obtained earlier, rather than being determined solely by the input at the current position. Its temporal update can therefore be abstracted as

$$S_t = AS_{t-1} + \mathcal{B}_t(x_t; K_{\leq t}, V_{\leq t}). \quad (6)$$

The main distinction between GSM and linear attention is therefore not whether they use a previous state, but whether constructing a new state can involve revisiting the explicit history. Linear attention can only further transform the compressed state, whereas GSM can retrieve historical information again based on new intermediate representations. Accordingly, GSM must retain an encoder-side history cache: it is the decoder state size, rather than the total historical storage of the full model, that remains fixed.

2.3 MULTI-STEP STATE UPDATES AND CHAIN-OF-THOUGHT

DeltaNet applies one rank-one state correction per input, with the expressiveness of each update constrained by its form. DeltaProduct, proposed by Siems et al., extends this process to multiple updates to increase the expressiveness of state transitions Siems et al. (2025). Experiments show improvements in state tracking, language modeling, and some in-context retrieval tasks, as well as substantially stronger length extrapolation. However, the number of updates is specified in advance, making it difficult to allocate computation dynamically according to the demands of different inputs. Increasing the number of updates also incurs substantial computational overhead. These limitations have hindered the broader adoption of multi-step state updates.

GSM faces a similar information bottleneck: although encoding involves multiple rounds of history retrieval, the new information at each position must ultimately be incorporated into a finite-dimensional hidden representation. This raises two questions: whether a single write can accommodate the information required by the current computation, and how to supplement the shared state when subsequent decoder computation creates new information needs. Chain-of-thought (CoT) offers a way to address this bottleneck by generating intermediate reasoning steps Wei et al. (2022). In GSM, each generated token passes through the encoder at the next step, providing another opportunity to retrieve historical information and write to the state. If the decoder expresses intermediate conclusions or new information needs in subsequently generated tokens, the encoder can adjust retrieval accordingly and incorporate additional information into the shared state. CoT can therefore distribute information acquisition and processing across multiple time steps, allowing the shared state to be continually updated around the current reasoning step without requiring a single write to anticipate all subsequent information needs.

DeepSeek-V4.1-Flash has the same state capacity as GSM in this comparison, since attention in both models operates over the same number of hidden representations. The difference is that DeepSeek-V4.1-Flash retains KV covering the history and allows some decoder layers to reselect entries using new queries. By contrast, the GSM decoder accesses only a fixed window and relies on subsequent encoding steps to incorporate additional historical information. Thus, once CoT addresses the information bottleneck described above, GSM should, in principle, be capable of matching the strong long-context processing capabilities of leading models.

3 MODEL ARCHITECTURE

As illustrated in Figure 1, GSM adopts a causal encoder-decoder architecture that separates historical information aggregation from layer-by-layer decoding. The encoder aggregates historical information through multiple retrieval stages and fuses the accumulated output with its final encoded representation to construct a shared KV state with a fixed window size. Each decoder layer accesses the same state using queries updated from the preceding layer, preserving computational dependencies across layers while avoiding repeated construction of historical KV and long-range indexing.

3.1 MULTI-STAGE HISTORY AGGREGATION

Let d denote the model’s hidden dimension and S the number of aggregation stages. Let $\mathbf{h}_t^{(s)} \in \mathbb{R}^d$ denote the encoder representation at position t in aggregation stage s , obtained by mixing the multi-stream residual representations of the corresponding layer Xie et al. (2025). Let $\mathbf{r}_t^{(s)}$ denote the accumulated retrieval output through that stage, initialized as $\mathbf{r}_t^{(0)} = \mathbf{0}$. The query input at each stage is

$$\mathbf{u}_t^{(s)} = \begin{cases} \mathbf{h}_t^{(1)}, & s = 1, \\ [\mathbf{h}_t^{(s)}; \mathbf{r}_t^{(s-1)}], & s > 1, \end{cases} \quad (7)$$

where $[\cdot; \cdot]$ denotes concatenation along the feature dimension. Queries are generated using stage-specific root mean square normalization (RMSNorm) Zhang & Sennrich (2019) and query projections:

$$\mathbf{q}_t^{(s)} = \text{Query}_s \left(\text{RMSNorm}_s(\mathbf{u}_t^{(s)}), p_t \right), \quad (8)$$

where p_t denotes the position metadata required for positional encoding. The input dimension is d in the first stage and $2d$ in subsequent stages, allowing later queries to depend on both the current encoded representation and previously retrieved information.

The aggregation branch directly reuses the local window $\mathcal{W}_t^{(s)}$, historical KV bank $\mathcal{B}^{(s)}$, and Top- K indices $\mathcal{I}_t^{(s)}$ used by the corresponding encoder layer, without a separate indexer. Retrieval and accumulation proceed as follows:

$$\Delta \mathbf{r}_t^{(s)} = \text{Output}_s \left(\text{Attn} \left(\mathbf{q}_t^{(s)}; \mathcal{W}_t^{(s)}, \mathcal{B}^{(s)}[\mathcal{I}_t^{(s)}] \right) \right), \quad (9)$$

$$\mathbf{r}_t^{(s)} = \mathbf{r}_t^{(s-1)} + \Delta \mathbf{r}_t^{(s)}. \quad (10)$$

Here, the aggregation query determines the attention weights over the selected entries; it does not reselect the Top- K entries. Multi-stage aggregation therefore progressively adjusts the retrieved information by updating queries while reusing the selections already made by the encoder.

The accumulated retrieval output is updated across encoding stages without being written back to the main encoder branch between stages. The encoder thus retains its original computational path across layers, and the aggregation result is fused with its output only after encoding is complete.

3.2 SHARED STATE CONSTRUCTION

Let $\mathbf{e}_t$ denote the mixture of the final multi-stream encoder representations. The model fuses it with the accumulated retrieval output and applies a shared KV mapping to generate a state entry:

$$\mathbf{z}_t = \mathbf{e}_t + \mathbf{r}_t^{(S)}, \quad (11)$$

$$\mathbf{m}_t = \text{KV}_{\text{sh}}(\mathbf{z}_t, p_t). \quad (12)$$

This mapping applies input normalization, KV projection, KV normalization, and positional encoding in sequence, producing one state entry per position without compression along the sequence dimension.

At position t , the decoder accesses only the shared KV entries corresponding to the most recent M valid positions:

$$\mathcal{M}_t = [\mathbf{m}_{\max(1, t-M+1)}, \dots, \mathbf{m}_t], \quad M = W + K, \quad (13)$$

where W and K denote the encoder’s local window size and number of sparsely retrieved entries, respectively. The actual accessible range is constrained by document boundaries and position masks. The decoder’s retrieval budget therefore matches that of a single encoder attention operation. This window limits only the decoder’s direct access: distant information has already been aggregated into the representations within the window by the encoder, allowing the shared state to carry both recent and long-range information.

The fixed-size shared window also allows encoder computation during prefill to be pruned according to its dependencies. For an input of length N that requires only next-token prediction, only the shared KV entries for the final $m = \min(N, M)$ positions need to be constructed. It is therefore unnecessary to process every position through every encoder layer. Let L_E denote the number of encoder layers and ℓ_* the last historical KV source layer, with layers indexed from 1. Once full-sequence KV construction at this layer is complete, each layer $\ell \geq \ell_*$ performs attention retrieval and subsequent within-block computation only for the final

$$R_\ell = \min(N, m + (L_E - \ell)(W - 1)) \quad (14)$$

positions. The additional positions cover the dependencies introduced by subsequent local windows, allowing the computation range to shrink progressively with depth until only the final m positions remain. Each aggregation stage likewise needs to compute retrieval outputs only for these positions. Finally, the model constructs the shared state from the fused representations at the final m positions and initializes the decoder with the representation at the last position, reducing prefill computation in the later encoder layers while preserving all computational dependencies.

3.3 SHARED STATE DECODING

The decoder initializes the representation at the current position with $\mathbf{z}_t$. The attention module in each layer retains only its own query projection, output projection, and attention sink; it no longer

Table 1: Downstream task performance (%). Metrics for tasks marked with $\dagger$ are specified in Appendix A.2.

Task	0.5B		1.5B		8B-A1.5B	
	Baseline	GSM	Baseline	GSM	Baseline	GSM
ARC-Easy † Clark et al. (2018)	51.85	51.64	59.09	58.59	72.90	71.55
ARC-Challenge †	28.75	26.88	33.45	33.36	36.69	36.69
PIQA † Bisk et al. (2020)	66.81	66.16	70.24	69.80	73.23	74.43
SciQ † Welbl et al. (2017)	74.10	73.10	81.90	82.40	89.80	89.80
BLiMP Warstadt et al. (2020)	77.01	79.34	80.18	80.79	81.66	82.24
HellaSwag † Zellers et al. (2019)	40.92	40.45	50.13	49.35	44.52	44.65
WinoGrande Sakaguchi et al. (2020)	53.43	52.17	54.62	54.14	55.41	57.14
MMLU Hendrycks et al. (2021)	25.82	26.23	25.47	26.41	25.75	27.10
OpenBookQA † Mihaylov et al. (2018)	34.40	33.00	37.60	36.60	29.00	28.80
LAMBADA Paperno et al. (2016)	32.62	32.74	41.84	41.12	46.79	46.30
Avg.	48.57	48.17	53.45	53.26	55.58	55.87

generates independent KV or performs long-range indexing. Let $\mathbf{d}_t^{(\ell-1)}$ denote the mixed representation entering the attention branch of layer ℓ . Then,

$$\mathbf{q}_{t,D}^{(\ell)} = \text{Query}_{D,\ell} \left(\text{RMSNorm}_{D,\ell}(\mathbf{d}_t^{(\ell-1)}), p_t \right), \quad (15)$$

$$\mathbf{a}_{t,D}^{(\ell)} = \text{Output}_{D,\ell} \left(\text{Attn}(\mathbf{q}_{t,D}^{(\ell)}; \mathcal{M}_t) \right). \quad (16)$$

All decoder layers at the same position access the same $\mathcal{M}_t$, but their queries change as representations are updated across layers. Sharing KV across layers therefore preserves sequential computational dependencies across depth. The shared state remains unchanged within the decoder, and no additional history retrieval is performed.

4 EXPERIMENTS

We evaluate whether GSM can maintain model performance while reducing the cost of accessing historical information. We construct a baseline based on the DeepSeek-V4.1-Flash architecture and conduct paired comparisons with GSM at matched model scales and training budgets. The baseline combines sparse attention, KV compression, and cross-layer sharing, allowing us to assess whether GSM offers further improvements over an already efficient attention architecture. Detailed experimental settings are provided in Appendix A.

4.1 DOWNSTREAM TASK PERFORMANCE

We evaluate 0.5B and 1.5B dense models and an 8B-A1.5B mixture-of-experts (MoE) model on ten downstream tasks covering commonsense reasoning, language understanding, and knowledge-based question answering. At each scale, we use the checkpoints obtained at the end of the main training phase. Avg. denotes the unweighted mean across the ten tasks.

As shown in Table 1, GSM achieves average performance comparable to the baseline at both dense model scales and a higher average score in the 8B-A1.5B MoE configuration. BLiMP and MMLU improve at all three scales, while the remaining tasks exhibit mostly small variations. These results indicate that the shared state aggregated by the encoder supports subsequent layer-by-layer computation, allowing GSM to maintain overall task performance while substantially simplifying history access in the decoder. The same trend holds for both dense and MoE models, suggesting that GSM scales across model sizes and feed-forward network (FFN) architectures.

4.2 LONG-CONTEXT MODELING

We evaluate long-context perplexity on a held-out FineWeb set using the 8B-A1.5B models after adaptation to 64K contexts with YaRN Peng et al. (2024). Given the strong task capabilities of

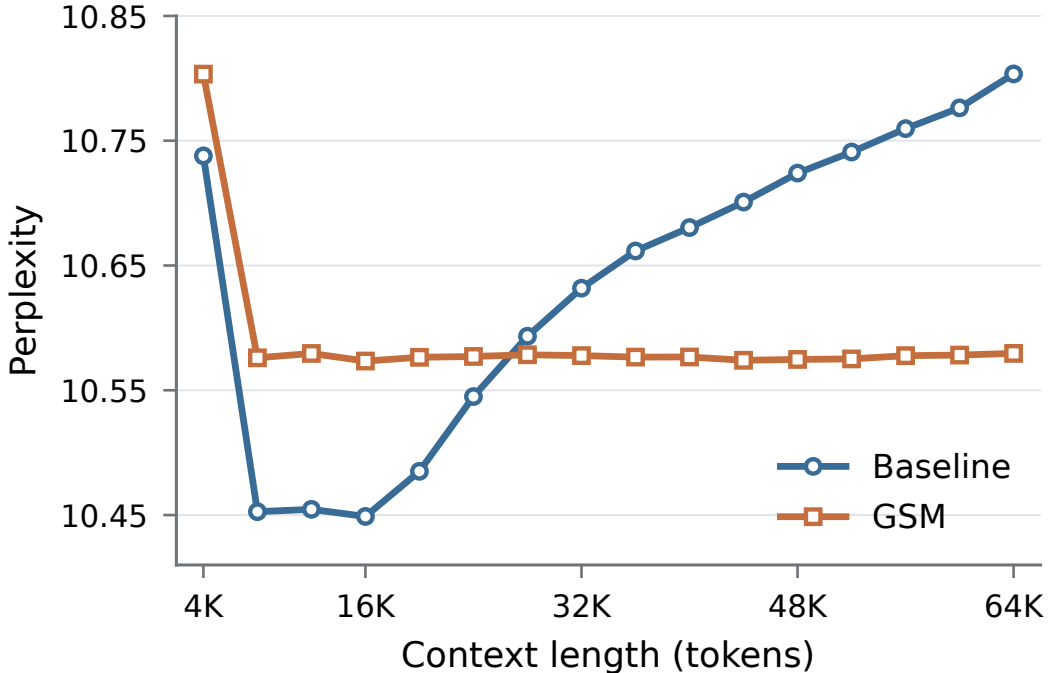


Figure 2: Long-context perplexity of the 8B-A1.5B models on FineWeb.

Table 2: RULER retrieval accuracy of 1.5B GSM (%).

Answer Format	4K	8K	16K	32K	64K	Avg.
Direct Answer	98.33	100.00	98.33	100.00	98.33	99.00
Short CoT	100.00	100.00	100.00	100.00	100.00	100.00

the 8B models, we additionally evaluate three RULER Hsieh et al. (2024) retrieval tasks using the smaller 1.5B GSM after task adaptation. The former assesses language modeling stability over long contexts, while the latter tests long-range information retrieval. For perplexity evaluation, we score the same target suffix at every context length to control for differences in evaluation positions.

Figure 2 shows that perplexity initially decreases for both models as the context expands, after which their trends diverge. The baseline achieves lower perplexity at intermediate lengths but gradually deteriorates with longer inputs. GSM follows a more stable trajectory and achieves lower perplexity in the long-input regime. Because the same target suffix is evaluated at every length, these results directly reflect the effect of additional context on prediction quality, indicating that GSM’s shared state can consistently use an expanding history. Together with the perplexity results over complete 64K sequences (Appendix A.2), these findings show that GSM maintains overall language modeling performance while exhibiting greater stability in long-context modeling.

Table 2 further shows that GSM maintains high retrieval accuracy with direct answers across 4K–64K contexts, without degradation as the context expands. Although the decoder accesses only the shared state within a fixed window, distant targets can still be retrieved by the encoder and aggregated into the current state. Thus, the fixed-size shared state does not prevent access to distant information; instead, it shifts long-range retrieval from the decoder to the encoder.

4.3 COMPUTATIONAL EFFICIENCY

To directly assess GSM’s computational complexity advantage, we benchmark the 1.5B dense models on the same H800 hardware. Training uses 4K sequences, while inference covers input lengths

Table 3: Training efficiency of the 1.5B models at a sequence length of 4K.

Model	s/step↓	tokens/s↑	GiB/GPU↓
Baseline	2.578	50,833	43.54
GSM	2.462	53,231	44.06

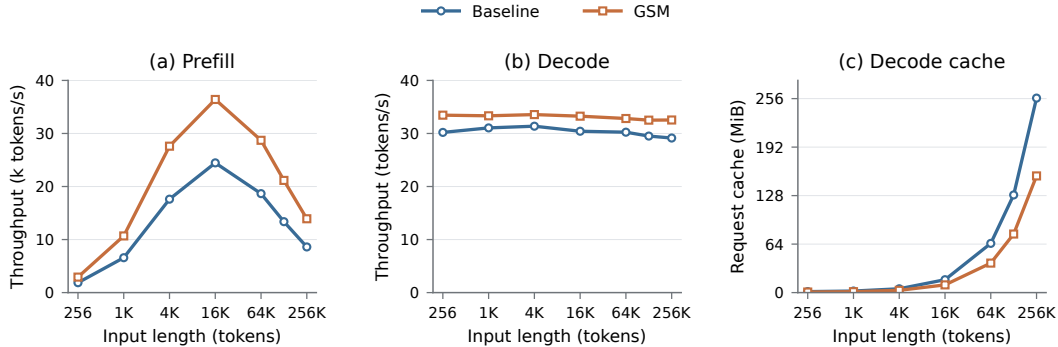


Figure 3: Prefill throughput, decoding throughput, and cache memory usage during decoding for the 1.5B models.

from 256 to 256K tokens with a fixed 64 decoding steps. The measured speed of the 8B-A1.5B MoE models is more sensitive to expert kernels and scheduling implementations. We therefore use dense models to facilitate a direct comparison of architectural overhead.

Table 3 shows that GSM achieves higher training throughput at 4K, with peak GPU memory usage comparable to the baseline. The shared state reduces the cost of history access in the decoder, improving overall training speed despite the additional aggregation performed by the encoder.

Figure 3 shows that GSM achieves higher prefill throughput at every tested length, with the advantage persisting as the input grows. GSM concentrates history retrieval and aggregation in the encoder, after which the decoder accesses only a fixed-size shared state. This reduces repeated historical selection, access, and cache construction. The results demonstrate that this computational structure translates directly into faster prefill for long sequences.

Decoding likewise exhibits a consistent throughput advantage alongside substantially lower cache memory usage. Because the decoder’s dependence on history is reduced to a fixed-window state, its layers no longer maintain and access long-sequence KV that grows with the context, reducing both computation and cache overhead during decoding. This advantage persists as context length increases, consistent with GSM’s fixed state window design.

4.4 ABLATION STUDIES

4.4.1 SHARED HISTORY RETRIEVAL

Using 0.5B models, we compare a separate aggregation indexer with reuse of the main attention Top- K selections. Both variants use the same training budget, accumulated retrieval output, and query formulation, allowing us to assess whether the aggregation branch requires independent history selection.

Table 4 shows that reusing the main attention Top- K selections largely preserves performance on standard tasks. Average RULER accuracy decreases slightly, while accuracy at 64K matches that of independent retrieval. Appendix A.4 shows that the overlap between the two sets of indices decreases markedly as the context grows, indicating that sharing changes which historical positions the aggregation branch accesses. The model nevertheless maintains comparable task performance, suggesting that aggregation can adapt to the candidate set provided by main attention without requiring a separate history index for the shared state.

Table 4: Shared retrieval ablation (0.5B, %).

Retrieval Method	Standard Avg.	RULER Avg.	RULER 64K
Separate Aggregation Indexer	64.918	96.67	93.33
Shared Main Attention Top- K	64.765	94.33	93.33

4.4.2 THE ROLE OF CoT IN LONG-CONTEXT INFORMATION USE

Starting from the same 1.5B GSM checkpoint adapted to 64K with YaRN, we train direct-answer and short-CoT variants using the same retrieval examples and number of training steps. Evaluation considers only the final answers. Detailed settings are provided in Appendix A.5.

As shown in Table 2, short CoT achieves 100% accuracy at every tested length from 4K to 64K, including 64K inputs that exceed the lengths used for task adaptation. Per-task results further show that the remaining direct-answer errors arise primarily from multi-key retrieval, and short CoT closes this gap. This finding is consistent with the analysis in Section 2.3: generating intermediate tokens produces new model states and provides additional computational steps for subsequent history retrieval and state updates, allowing retrieval, verification, and answering to be distributed across multiple time steps. Multi-step generation can therefore further improve GSM’s use of long-range information in the shared state.

5 CONCLUSION

We introduced GSM, which concentrates the selection and aggregation of historical information in the encoding stage and supports layer-by-layer decoder computation through a shared state with a fixed window size. By reusing aggregated historical representations across layers, GSM reduces repeated history selection and cache overhead during decoding while preserving access to long-range information and computational depth. Experiments show that shared states improve computational efficiency while maintaining model performance and long-context modeling capabilities, offering an effective architectural approach to reducing the cost of processing history in long-sequence modeling.

REFERENCES

- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/9e23d020c18e4c40d81c6a0fc7a46f68-Abstract-Conference.html.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL <https://arxiv.org/abs/1803.05457>.
- DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026a. URL <https://arxiv.org/abs/2606.19348>.
- DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. *arXiv preprint arXiv:2609.19969*, 2026b. URL <https://arxiv.org/abs/2609.19969>.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=d7KBjmI3GmQ>.

- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract-Conference.html.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5156–5165. PMLR, 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- Kimi Team. Kimi K3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL <https://arxiv.org/abs/2607.24653>.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2381–2391. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1260. URL <https://aclanthology.org/D18-1260/>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://proceedings.neurips.cc/paper/2022/hash/blefde53be364a73914f58805a001731-Abstract-Conference.html>.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534. Association for Computational Linguistics, 2016. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144/>.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/874a4d89f2d04b4bcf9a2c19545cf040-Abstract-Conference.html.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6399>.

- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9355–9366. PMLR, 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.
- Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. DeltaProduct: Improving state-tracking in linear RNNs via householder products. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.nips.cc/paper_files/paper/2025/hash/elea2520fbf9cd1600b287dde67e0a3c-Abstract-Conference.html.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohanney, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020. doi: 10.1162/tacl.a.00321. URL <https://aclanthology.org/2020.tacl-1.25/>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://papers.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pp. 94–106. Association for Computational Linguistics, 2017. doi: 10.18653/v1/W17-4413. URL <https://aclanthology.org/W17-4413/>.
- Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. mHC: Manifold-constrained hyper-connections. *arXiv preprint arXiv:2512.24880*, 2025. URL <https://arxiv.org/abs/2512.24880>.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472/>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.

Table 5: Main training configurations.

Scale	Layers	FFN	Input Tokens
0.5B	24	Dense	10B
1.5B	28	Dense	15B
8B-A1.5B	28	MoE	18B

Table 6: Supplementary FineWeb perplexity results for the 8B-A1.5B models.

Evaluation Setting	Baseline	GSM
Main Training Held-Out Set	9.82440	9.94952
Full 64K After Adaptation	9.66497	9.66827

A EXPERIMENTAL SETTINGS AND SUPPLEMENTARY RESULTS

A.1 MODELS AND TRAINING

The baseline retains sparse and compressed attention, sliding windows, cross-layer sharing, and manifold-constrained hyper-connections (mHC) from the DeepSeek-V4.1-Flash text backbone. At each model scale, the baseline and GSM use matched network depths, hidden dimensions, training data, and optimization settings. GSM uses a fixed shared state window of $M = 256$. The main training phase uses a sequence length of 4,096 and a global batch size of 128. Training budgets are measured in input tokens, with configurations summarized in Table 5.

The 0.5B and 1.5B models have hidden dimensions of 1,024 and 2,048, respectively, and FFN intermediate dimensions of 4,608 and 6,144. Half of the layers are assigned to the encoder and half to the decoder. Aggregation stages are placed at layers 6, 10, and 12 in the 0.5B model and at layers 6, 10, and 14 in the 1.5B model. Both dense models use an encoder local window size of 128 and a sparse retrieval budget of 128 entries. For the 8B-A1.5B configuration, the baseline and GSM have 8.001B and 8.033B total parameters, respectively, with 1.497B and 1.529B active parameters.

A.2 TASK EVALUATION AND LONG-CONTEXT SETTINGS

Downstream tasks are evaluated using lm-evaluation-harness. MMLU uses a 5-shot setting, while all other tasks use a 0-shot setting. The maximum context length is 4,096, and the evaluation seed is 42. Tasks marked with † in Table 1 use `acc_norm` for dense models and `acc` for MoE models; all remaining tasks use `acc`. For BLiMP and MMLU, subtask scores are first aggregated by sample count, after which the resulting task scores are averaged equally with those of the other tasks.

Both 8B-A1.5B models undergo 100 adaptation steps on FineWeb at a sequence length of 64K, using a YaRN factor of 16 and approximately 0.21B additional input tokens per model. Perplexity (PPL) is evaluated on a held-out FineWeb set whose source files are disjoint from those used for training. The evaluation contains 16 long windows constructed by continuously concatenating web documents, following the training procedure. We evaluate context lengths from 4K to 64K in increments of 4K. At every length, we score the same 4,096 target tokens at the end of each sequence, yielding 65,536 targets in total. PPL is computed from the mean negative log-likelihood over all target tokens.

We additionally score all 1,048,576 targets across the complete 64K sequences and evaluate the main training checkpoints on a separate sample set of the same size. Results are reported in Table 6.

RULER is evaluated on 1.5B GSM using three retrieval tasks: `niah_multikey_1`, `niah_single_1`, and `niah_single_2`. Each task is evaluated on 20 examples at each context length of 4K, 8K, 16K, 32K, and 64K, yielding 300 test examples in total. The direct-answer and short-CoT variants use identical test sets and are scored solely on their final answers. Each per-length result in Table 2 is the mean accuracy across the three tasks, and Avg. is the unweighted mean across the five lengths. Table 7 further reports each task’s average accuracy across the five lengths.

Table 7: Per-task RULER accuracy of 1.5B GSM (%).

Task	Direct Answer	Short CoT
niah_multikey_1	97.00	100.00
niah_single_1	100.00	100.00
niah_single_2	100.00	100.00

Table 8: Overlap between aggregation and main attention indices (%).

Length	Stage 1	Stage 2	Stage 3
4K	43.008	51.139	55.604
16K	28.478	34.225	43.645
32K	23.445	28.565	39.441
64K	19.023	24.676	36.170

A.3 EFFICIENCY MEASUREMENT

Training efficiency for the 1.5B models is measured in separate, sequential runs on the same eight H800 80GB GPUs. The sequence length is 4,096, with a per-GPU batch size of 4 and a global batch size of 32. Both models use identical activation recomputation settings. After five warm-up steps, we measure 12 consecutive steps and compute aggregate throughput across all eight GPUs from the median step time. Timing includes the forward pass, backward pass, gradient communication, and optimizer update. GPU memory usage is reported as the maximum peak allocated memory across the eight GPUs.

Inference uses the final weights from the main training phase, with paired measurements on the same H800 80GB GPU using bfloat16 (BF16), unquantized caches, and a batch size of 1. For each input length, we perform two warm-up runs and three measured runs, reporting the median. Each run continues with exactly 64 teacher-forced decoding steps after prefill. Prefill throughput is defined as the number of input tokens divided by the time required to produce the first output logits and prepare the cache. Decoding throughput is defined as 64 divided by the total time for the continuation. Timing includes model computation, vocabulary projection, and request state maintenance.

Cache memory usage during decoding is measured after the 64-step continuation by summing the unique underlying storage allocations occupied by the request cache, counting shared storage only once. Inputs of 128K and 256K tokens are constructed by cyclically repeating corpus windows to assess efficiency at longer input lengths. Both models use on-demand prefill, KV reads restricted to selected entries, and CUDA Graphs. Numerical consistency is verified for prefill, continuation with cached states, compression boundaries, and document boundaries.

A.4 SHARED RETRIEVAL ABLATION

The shared retrieval ablation compares two 0.5B GSM variants. Both use the same accumulated retrieval output and concatenated query inputs. One uses a separate aggregation indexer to select historical entries, while the other directly reuses the main attention Top- K selections from the corresponding encoder layer. Both models undergo main training on approximately 10B tokens, 200 adaptation steps at 64K, and identical task adaptation with direct answers. Standard Avg. is the unweighted mean over ARC-Easy, PIQA, SciQ, BoolQ, and BLiMP. The first three tasks use `acc_norm`, and the remaining two use `acc`.

To examine differences in the historical positions selected by the two retrieval mechanisms, we compare the aggregation indices with the corresponding main attention indices within the model that uses a separate indexer. Top- K overlap is computed as the intersection size divided by K , considering only queries with more than K valid candidates. Results are reported in Table 8.

The overlap decreases consistently as context length increases, indicating that the separate indexer and main attention select increasingly different historical positions. Together with the task results in Table 4, this shows that sharing the main attention Top- K selections preserves comparable model

performance despite substantially changing the candidate sets. A separate aggregation indexer is therefore not necessary for constructing the shared state.

A.5 CoT ABLATION

The CoT ablation starts from the same 1.5B GSM checkpoint adapted to 64K with YaRN and trains two variants: direct answer and short CoT. Both use the same 512 RULER retrieval training examples and are trained for 64 steps. Training lengths cycle through 4K, 8K, 16K, and 32K, while 64K is reserved exclusively for length extrapolation evaluation.

The direct-answer variant receives supervision only on the final answer. The short-CoT variant includes a brief retrieval and verification process in the target response and uses `Final answer:` to mark the final answer. Both variants use identical initial weights, input examples, and numbers of update steps, and evaluation considers only the final answers. Short CoT contains more supervised tokens, allowing the model to learn retrieval, verification, and answering across multiple generation steps. It achieves 100% accuracy at every tested length and on all three RULER tasks, as detailed in Tables 2 and 7.